Infectious Disease Simulation

Connor Feng

EID: ctf664
TACC Username: connorfeng
Email: connorfeng@utexas.edu

1 Introduction

In recent times, the spread of infectious diseases and vaccines have been a trending topic in US politics and among Americans. COVID-19 and its effects have completely changed people's perception of infectious diseases, and the topic has become quite polarizing. Robert F. Kennedy Jr, the current US Secretary of Health and Human Services, said in a Fox News interview in 2023 that "autism comes from vaccines"[1]. There have also been recent trends for parents in Western countries refusing to vaccinate their children due to perceived fears [2], which is termed the anti-vaccination movement. Regardless of politics and social issues, knowing how an infectious disease spreads and its consequences is important to understand and can be extremely educational. In my project, I will ultimately model how fast a disease spreads through a partially vaccinated population, depending on various parameters. My simulation adopts a agent-based modeling approach, where I represent a "Person" with specific behaviors and rules that interact with each other in a "Population". With this model, I addressed these questions:

1. How does the vaccination rate affect how long it takes for a disease to run through a population, with varying probabilities of transmission?

2. What is the percentage of people that need to be vaccinated to achieve herd immunity, with varying probabilities of transmission?

3. How do mutations affect the vaccination rates required to subdue a disease?

4. Some people refuse to get vaccinated because they believe that if only a few people bend the rules, then they will not bear the consequences. Is this true?

By running experiments that aim to answer these questions, I came up with the conclusions that a disease can spread among a population very fast if not managed properly, and that it is important for everyone to do their due diligence and get vaccinated for their safety and others, as there is very little room for people be not vaccinated.

2 Code Structure

For my simulations, my source code has 4 main classes:

- Disease
- Person
- Population
- Generator

2.1 Disease Class

A disease in the disease class has 3 main attributes:

- Transmission probability - The probability of the disease infecting another person, this is the contagiousness
- Duration - This is how long the disease stays in a person's system for.
- Variant ID - Used for mutations, this is the specific variant of the disease.

The disease class is used to keep track of mutations as well, as there are global variables to keep track of the current strand of the disease and also the number of transmissions a disease has had. There are also methods for the disease to mutate and record the number of transmissions.

2.2 Person Class

A person in the person class has these attributes:

- Status - to keep track of whether the person is susceptible, sick, vaccinated, or recovered. An enum class was used.
- Disease - The disease that a person has (if any). The optional class template was used so the value can be null when need be.
- Days Sick - Keeps track of the days that a person has been infected with a disease. Helpful for a person to recover eventually.
- Variant IDs - A vector of integers that represent the disease strands a person has been infected with before. Useful for keeping track of infections with mutations.

The methods of the person class are for interactions between two people and infecting/vaccinating people.

2.3 Population Class

The population class essentially is a vector of people that is represented in a matrix. For my code I chose to make a one dimensional vector that represents a stretched out matrix, and I performed calculations that allow me to get a specific row and column of the matrix.

The population class has methods that are related to a person spreading a disease to its neighbors or random people. There is also a method that can display the population using the format class template. Below is a sample output of the display of a population.

```
? ? - + + + - - + -
- - - + + + + + + +
+ - - + + ? - - + +
+ - + - - - - - - +
- - - ? - + - - - ?
- + - + - + + ? - ?
+ - + + - + - ? - -
- + - + - + - - - ?
+ + ? + + - - - + +
- - ? - - ? ? - - +
======Day: 3======
```

The "?" represents susceptible people, the "-" represents people who are recovered or vaccinated, and the "+" represents infected people.

2.4 Generator

The generator class is used for random number generation. I followed the instructions from section 24.7 of the textbook. My methods can create a random double from 0 to 1, create a random int from 0 to a specified range, and can shuffle a given vector's indices around.

2.5 Sanity Tests

In the tests folder, there is a script that performs sanity tests on my code using the catch2 library. This ensures that my simulations are as accurate as possible and the logic surrounding my code makes sense.

2.6 Experiments

In my experiments folder, I perform 3 experiments which will all be talked about in the later sections. Experiments 1,2, and 3 correlate to sections 3,4, and 5, respectively, in this report.

3 Vaccination Rate's Affect On Disease Spreading

In this experiment, I aim to find the relationship between how long a disease runs through a population and its vaccination rate. For context, how long a disease runs through a population is the number of the days from the start until the number of infected people drops to zero.

3.1 Experiment Parameters

For this experiment, I established these variables as constants:

- Population = 10,000 people
- How long a disease lasts = 7 days
- Number of random people contacted per day (not including neighbors) = 5 people

These are the variables that I change:

- Vaccination rate
- Probability of disease transmission at contact

Here is a code snippet of how I gather one of my data points for this experiment.

```
Population UT(100);
Disease flu(.1, 7);
UT.random_vaccination(.1);
UT.random_infection(flu);
int time = 0;
while(UT.num_sick()>0)
{
    UT.contagion();
    UT.spreading(5);
    UT.one_more_day();
    time ++;

    cout << "Number of people infected after day: " << time << endl;
    cout << UT.num_sick() << endl;
}
cout << "The disease took " << time << " days to run" << endl;
```

As you can see from my code, I create a population that has 100 rows and 100 columns, which is equal to 10,000 elements(people). I also created a "Disease" object with a transmission probability of 10 percent and a duration of 7 days. "random_vaccination" vaccinates a specified number of random people. I then call the "random_infection" method in the population, which infects a singular random person in the population, creating a patient zero. I create a while-loop to keep progressing through days until the disease has ran through the population, which is when there is no one infected. Inside the for loop, I call "contagion" which contacts every person with their neighbors, and also "spreading" which contacts a number of random people in the population. I then print the number of infected people after every day for a sanity check, and I recorded the time it took for the disease to run through the population. Below is a sample output from my experiment:

```
Population: 10k, Transmission Probability: .1, Duration: 7 days, Contacts: 5
Vaccination Rate 0
Duration of the disease 32.48 days
Vaccination Rate 5
Duration of the disease 33.54 days
Vaccination Rate 10
Duration of the disease 34.83 days
```

3.2 Data Table

For each vaccination rate, I ran the simulation 100 times and took the average to account for the randomness.

Vaccination Rate vs. Disease Duration			
	Avg Duration (Days)		
Vaccination Rate	P = 0.1	P = 0.2	P = 0.3
0%	32.47	19.75	15.64
5%	33.28	20.48	16.32
10%	34.54	21.20	16.65
15%	36.85	21.87	17.19
20%	37.27	22.80	17.81
25%	39.21	23.88	18.64
30%	42.23	24.8	19.06
35%	45.03	26.48	20.25
40%	46.98	27.94	21.23
45%	52.21	29.61	22.36
50%	53.25	31.42	23.85
55%	60.46	34.06	25.4
60%	64.41	37.35	27.39
65%	65.66	39.21	30.54
70%	68.18	45.96	33.85
75%	73.74	51.93	39.29
80%	27.72	57.43	45.36
85%	11.82	70.27	52.74
90%	10.07	17.73	58.82
95%	7.87	9.19	9.69
100%	0.0	0.0	0.0

3.3 Graph

For the graphs I used python's matplotlib.

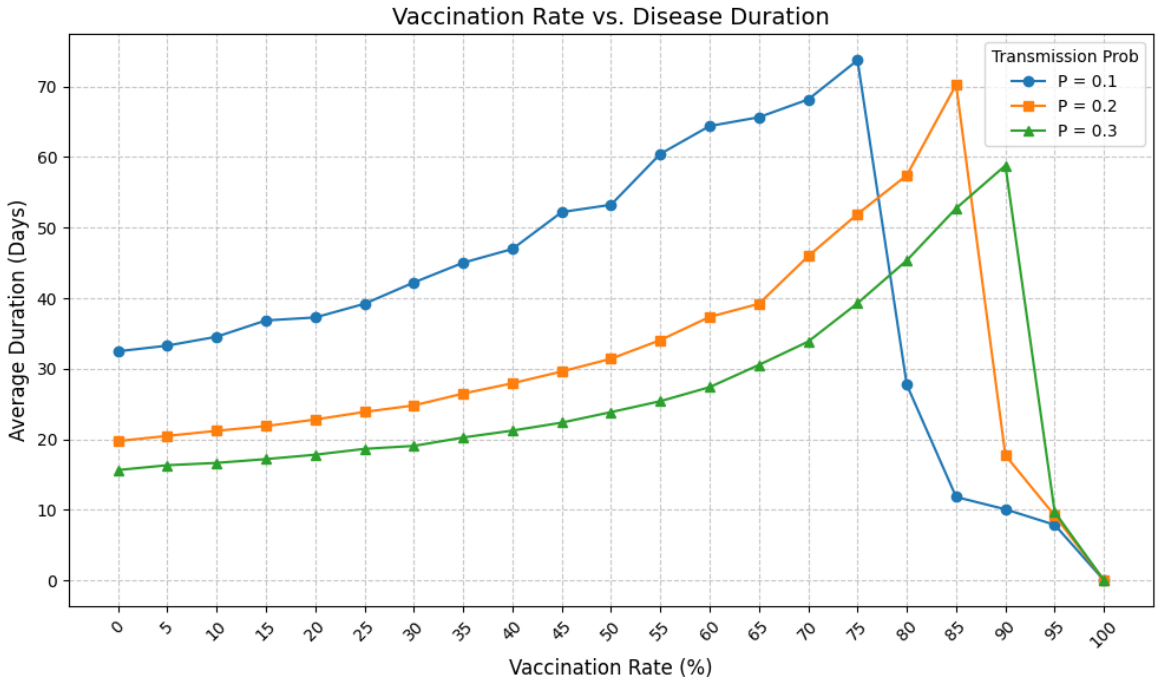


Figure 1: Disease duration as a function of the vaccination rate, with multiple lines for varying transmission probabilities for a disease

3.4 Analysis of Results

The results that I got from my experiment were very interesting. Here are some conclusions we can make from our data.

When the vaccination rate is not high, the duration of a disease increases as the vaccination rate increases

This seems very counterintuitive, because people typically would think that a higher vaccination rate means that less people get infected so the disease doesn't last longer. However, the effect we see from increasing vaccination rates is that the spread of the disease slows down because the vaccinated people act as "obstacles". This similar to someone traversing through a open field versus a maze. Thus, the disease has a longer duration.

More infectious diseases tend to not last as long

A disease that is more infectious progresses through the population faster, which causes the number of susceptible people to dwindle faster.

There is a specific vaccination rate for each transmission probability where the duration crashes

The duration of the disease can be seen to crash for transmission probabilities .1, .2 and .3 at vaccination rates of about .75, .85, .90 respectively. The disease has reached a point to where it can no longer sustain itself within the population, because the number of susceptible people is too little. At this point the disease is mostly subdued. This is known as herd immunity, and the vaccination rates required to achieve it is not trivial. For a disease with just a transmission probability of .3, a vaccination rate of at least 90% is needed to subdue the disease.

4 Herd Immunity Based On Transmission Probability

The concept of herd immunity within a population is achieved when enough people are vaccinated to the point where the people who aren't vaccinated will have a very low chance of getting sick. In this experiment, I aim to find the vaccinations rate required to achieve this herd immunity, and I want to find how the infectiousness of a disease affects these results.

4.1 Experiment Parameters

For this experiment I established the same constants as the previous experiment:

- Population = 10,000 people
- How long a disease lasts = 7 days
- Number of random people contacted per day (not including neighbors) = 5 people

The only variable we will be changing is the transmission probability. We will define that herd immunity is achieved at the vaccination rate when over 95% of the non-vaccinated population will never get sick.

Below is a code snippet of how I get the vaccination rate where herd immunity is achieved.

```
for(int i =99; i>=0; i-=1)
{
    cout << "Vaccination Rate: " << i << endl;
    double average = 0.0;
    for(int j = 0; j<100; j++)
    {
        // 100 * 100 = 10,000
        Population UT(100);
        Disease flu(.1, 7);
        double rate = i/100.0;
        UT.random_vaccination(rate);
        UT.random_infection(flu);
        double initial_susceptible = (double)UT.num_susceptible();
        while(UT.num_sick()>0)
        {
            UT.contagion();
            UT.spreading(5);
            UT.one_more_day();
        }

        double final_susceptible = (double)UT.num_susceptible();
        average += final_susceptible/initial_susceptible;
    }

    average = average/100.0;
    cout << average << " of non vaccinated people never got the disease" << endl;
    if(average<.95)
    {
        break;
    }
}
```

My code essentially creates a disease and a population with a patient zero, and my code then records the number of people who are "SUSCEPTIBLE" before and after the disease has taken its course, and then computes the ratio. My code begins with a vaccination rate of 99% and continues going down

by 1% until less than 95% of the non-vaccinated population doesn't get sick. I run the simulation 100 times for each vaccination rate and compute the average to account for randomness from the random number generator. Below is a sample output for this experiment:

```
Population: 10k, Duration: 7 days, Contacts: 5
Transmission Probability 0.05
Vaccination Rate 0.56
0.948487 of non vaccinated people never got the disease
```

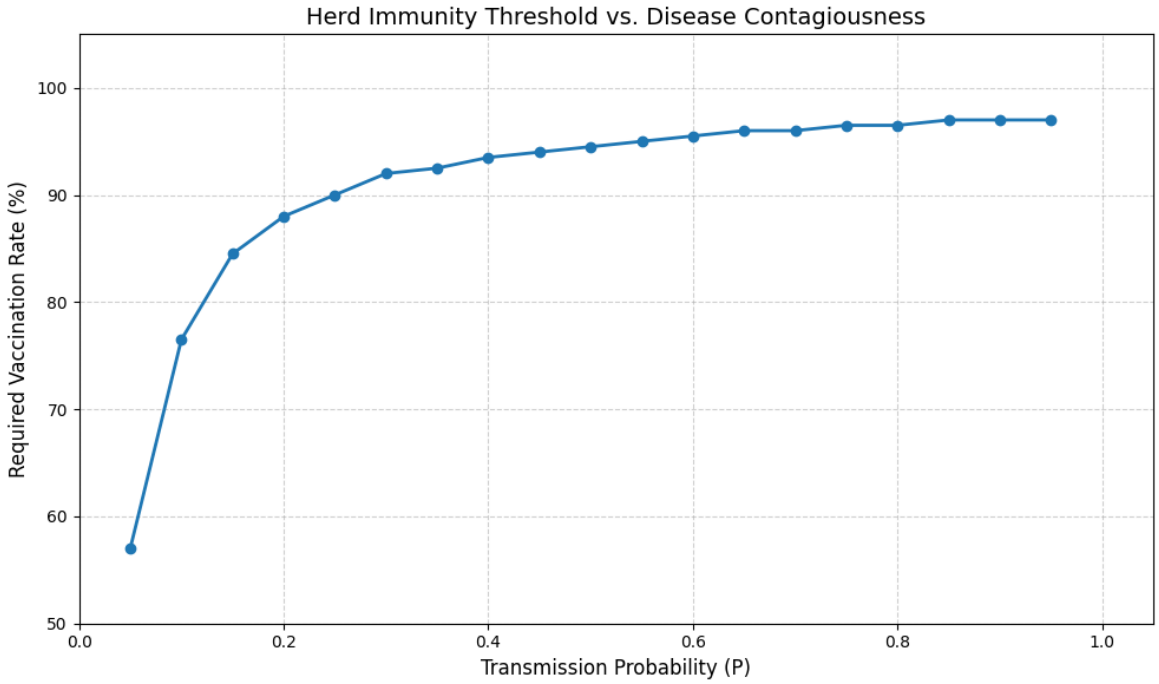
4.2 Data Table

Herd Immunity Thresholds (Vaccination required for 95% safety of the non-vaccinated)	
Transmission Probability	Required Vaccination Rate
.05	57.0%
0.1	76.5%
0.15	84.5%
0.2	88.0%
0.25	90.0%
0.3	92.0%
0.35	92.5%
0.4	93.5%
0.45	94.0%
0.5	94.5%
0.55	95.0%
0.6	95.5%
0.65	96.0%
0.7	96.0%
0.75	96.5%
0.8	96.5%
0.85	97.0%
0.9	97.0%
0.95	97.0%
1.0	100%

Table 1: The relationship between disease contagiousness and the vaccination rate required to achieve herd immunity.

4.3 Graph

The graph below shows the relationships between vaccination rates required for herd immunity vs the transmission probability of a disease. I used python's matplotlib library to create the plots.



4.4 Analysis of Results

Based on the table and plot that I created, we can make some conclusions regarding our data.

The relationship between contagiousness and the vaccination rate for herd immunity appears to be logarithmic

On the left side of the plot ($P < .3$), the function increases very sharply, but as P increases, the slope decreases and eventually starts to plateau at around a vaccination rate of 97%. Once a disease becomes very contagious ($P > .5$) the vaccination rate required for herd immunity essentially stays the same.

Achieving Herd Immunity Is Not A Trivial Task

The vaccination percentage to achieve herd immunity for highly contagious disease is very high. For every 100 people, at least 95 of them need to be vaccinated to diseases with a transmission probability below 55%.

5 Mutations And Its Effects

For this project, I wanted to incorporate disease mutation in my simulation, as disease mutation is very common and can greatly vary how long a disease can last in a population. The flu, for example, mutates and changes every year, meaning that you can get the flu more than once in your lifetime. This is why every year it is recommended to take the updated flu shot. In this experiment, I aim to figure out the vaccination rates required to fully subdue a disease that mutates. I will do this by analyzing the relationship between the vaccination rate and the mutation threshold where the disease takes over and lasts forever.

5.1 Mutations In Code

To simulate mutations, I modeled as follows.

- Every so many transmissions, a virus will mutate into a new variant
- A person who has recovered from one variant is still susceptible to other variants.
- Assume that each variant leaves a person sick the same number of days, and vaccination is all-or-nothing
- Having recovered from one variant is not protection against others

To do this I added some global variables to the Disease class.

```
        double transmission_prob;
        int duration;
        int variant_id;

        static inline int current_variant = 0;
        static inline int transmissions = 0;
        static inline int mutation_threshold = 1000;
```

The "current_variant" variable keeps track of the variant that entire disease thread is on. The value updates whenever it mutates. This is helpful because when I declare a new disease object, the "variant_id" becomes whatever the "current_variant" is. The "transmissions" variable keeps track of the number of people a disease strand has infected, and will update every time someone is infected. The value returns to zero whenever it reaches the "mutation_threshold".

I also added new methods to the Disease class to create a mutation

```
void Disease::record_transmission()
{
        transmissions++;
        if (transmissions >= mutation_threshold)
        {
                mutate();
        }
}

void Disease::mutate()
{
        current_variant++;
        transmissions = 0;
}
```

These methods just ensure that every time someone is infected, the number of transmissions is updated and that a mutation occurs when it needs to (whenever the mutation threshold is reached).

In addition, I also changed how contact between two people occur

```
void Person::contact_mutation(Person& new_person)
{
        if((status == health_state::SUSCEPTIBLE||status == health_state::RECOVERED)
&& new_person.is_infected())
        {
                Disease dis = new_person.disease.value();

                if(status == health_state::SUSCEPTIBLE||std::count
(variants_recovered.begin(),variants_recovered.end(),dis.get_variant_id()) == 0)
                {
                        double random = generator::random_double();
                        if(random < dis.get_transmission_prob())
```

```
                        {

                                get_infected(dis);

                        }
                }
        }
}
```

The interaction is different without mutations because now with mutations, a recovered person can still be infected by a different variant. Therefore my code checks if a person is recovered but has never gotten sick from the specific variant_id, which is done by have a person store a vector of ints that represent the variants it's been infected with.

5.2 Experiment Parameters

The parameters established for this experiment are very similar to previous experiments. I established these variables as constants:

- Population = 10,000 people
- How long a disease lasts = 7 days
- Number of random people contacted per day (not including neighbors) = 5 people

These are the variables that I change:

- Vaccination rate
- Probability of disease transmission at contact

5.3 Code For This Experiment

For this experiment, my objective is to find the mutation threshold value at which the disease would last for an infinite amount of time. Below is a code snippet of the experiment

```
for(int i = 5; i<=100; i+=5)
        {
                cout << "Vaccination Rate " << i << endl;
                int mutation = 0;
                for(int j = 10000; j>=0; j-=50 )
                {
                        Disease::set_mutation_threshold(j);
                        Population UT(100);
                        Disease flu(.3, 7);
                        double rate = i/100.0;
                        UT.random_vaccination(rate);
                        UT.random_infection(flu);
                        int time = 0;
                        while(UT.num_sick()>0 && time<=500)
                        {
                                UT.contagion();
                                UT.spreading(5);
                                UT.one_more_day();
                                time ++;
                        }
                        if(time>500)
                        {
                                mutation = j;
```

```
                                break;
                        }
                }
                cout << "mutation threshold where the disease wins " << mutation
<< endl;
        }
```

My code essentially attempts to get the duration of the disease with varying vaccination rates, and the code will either print out the threshold where the disease is running for longer than 500 days which we can interpret as the disease running on forever, or the disease will print out 0 which means that no mutation_threshold can successfully live in the population forever. Below is a sample output from this experiment:

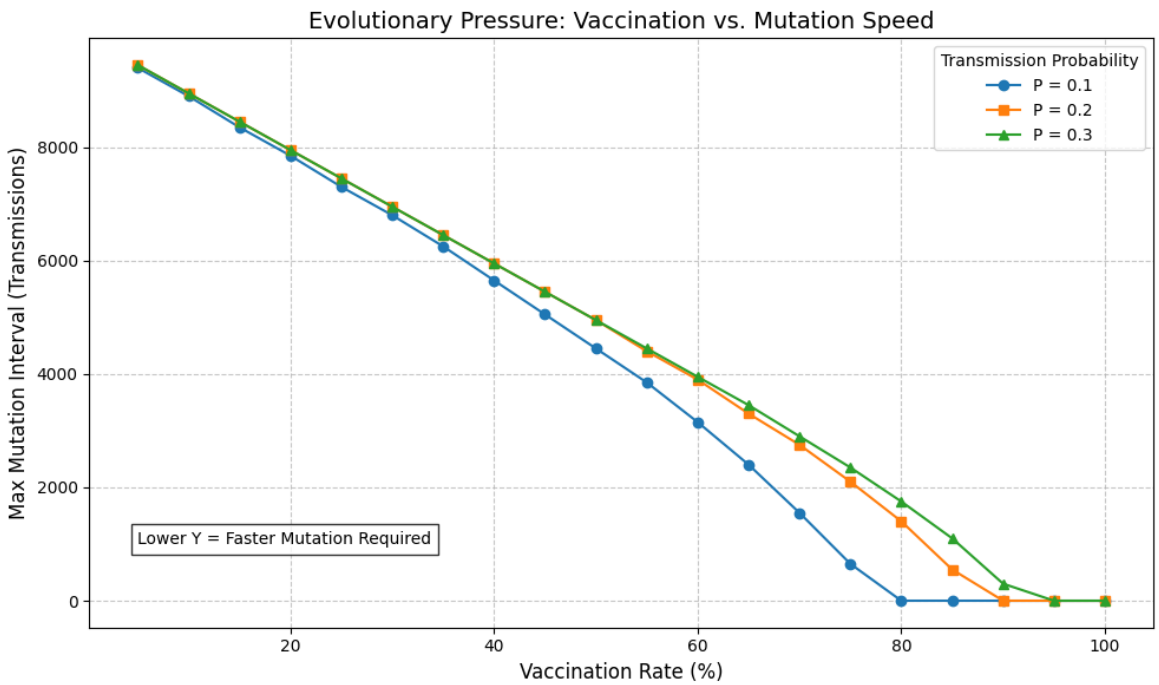
```
Population: 10k, Transmission Probability: .3, Duration: 7 days, Contacts: 5
Vaccination Rate 5
mutation threshold where the disease wins 9450
Vaccination Rate 10
mutation threshold where the disease wins 8950
Vaccination Rate 15
mutation threshold where the disease wins 8500
```

5.4 Data Table

Evolutionary Pressure: Vaccination vs. Mutation (Max mutation interval allowing endemic survival)			
	Transmission Probability		
Vaccination Rate	$P = 0.1$	$P = 0.2$	$P = 0.3$
5%	9400	9450	9450
10%	8900	8950	8950
15%	8350	8450	8450
20%	7850	7950	7950
25%	7300	7450	7450
30%	6800	6950	6950
35%	6250	6450	6450
40%	5650	5950	5950
45%	5050	5450	5450
50%	4450	4950	4950
55%	3850	4400	4450
60%	3150	3900	3950
65%	2400	3300	3450
70%	1550	2750	2900
75%	650	2100	2350
80%	0	1400	1750
85%	0	550	1100
90%	0	0	300
95%	0	0	0
100%	0	0	0

Table 2: The critical mutation threshold required for a disease to stay in the population forever. Lower numbers indicate the virus must evolve more rapidly to survive higher vaccination rates. A value of 0 indicates the disease goes extinct regardless of mutation speed.

5.5 Plot



5.6 Data Analysis and Interpretation

According to our data we can make this conclusion.

In order to fully subdue a disease, the vaccination rate needs to be high

For diseases with transmission probabilities of of $.1, .2, .3$, vaccination rates of $80\%, 90\%, 95\%$ respectively. Although a disease is not likely to mutate as quickly as our simulation, we also worked with only a population of $10,000$ people. These results mean that in order for a population to be safe from a disease and its mutations, the vaccination rates need to be very high. Keep in mind we are assuming that the vaccines are perfect, and will protect a person from every mutation. This is not realistic at all and just goes to show how mutations can really change the progression and duration of a disease.

6 The Big Picture

Based on our results, making sure that everyone is vaccinated is crucial for the safety of an individual and a population's safety.

Let's look at the most recent large scale pandemic to According to the World Health Organization, a mere 82% of the US population received at least one dose of a COVID-19 vaccine. This is nowhere near the proper vaccination rate to achieve herd immunity in a normal situation, which is why at least 100 million confirmed cases of COVID-19 with more than 1.2 million confirmed deaths[3]. In order to ensure that the next pandemic won't be as harmful as COVID-19, it is absolutely crucial for everyone to get vaccinated, as there only can be a very, very small population that can be non-vaccinated and still not affected by the disease.

The argument that people refuse to get vaccinated because they believe that if only a few people bend the rules, then they will not bear the consequences is very flawed according to my simulations. In the experiments from sections 3 and 4, I have shown that the vaccination rates required for the safety of the population are very high. If 5 out of 100 people were to have the mindset of being lazy to

get vaccinated, it could bear consequences on the entire population. Keep in mind that my simulation assumes that vaccines are perfect, which is nowhere near the truth. Many people who were vaccinated for COVID-19 still got infected. In reality, by not being vaccinated, one still puts vaccinated people at risk and particularly the immunocompromised and elderly.

So for my final closing words, GET VACCINATED!!!

References

[1] BBC Verify Team. "RFK Jr: Fact-Checking His Views on Health Policy." *BBC News*, 15 Nov. 2024, `www.bbc.com/news/articles/c0mzk2y41zvo`.

[2] Hussain, Azhar, et al. "The Anti-Vaccination Movement: A Regression in Modern Medicine." *Cureus*, vol. 10, no. 7, 3 July 2018, p. e2919. `https://doi.org/10.7759/cureus.2919`.

[3] World Health Organization (WHO). "COVID-19 Vaccines — WHO COVID-19 Dashboard." *Data-dot*, 31 Dec. 2023, `data.who.int/dashboards/covid19/vaccines`.